SEMANA 2 · PARTE 1

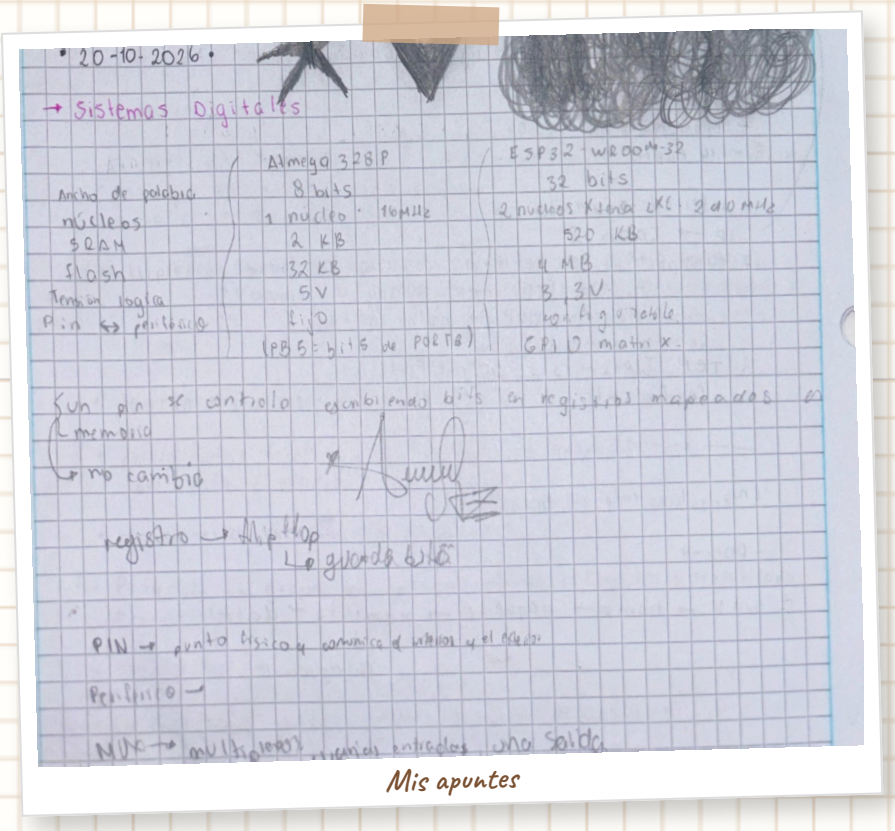
Ivett Anahi Zaragocin Mora

Del AVR al ESP32

PINES, REGISTROS Y EL PROGRAMA QUE NUNCA TERMINA

ATmega328P vs ESP32

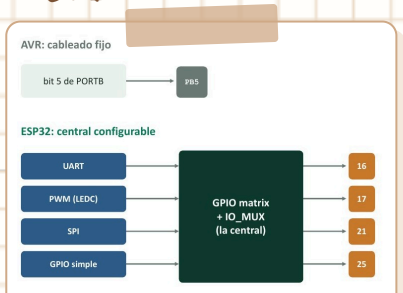
Característica	ATmega328P (semana 1)	ESP32-WROOM-32
Ancho de palabra	8 bits	32 bits
Núcleos / reloj	1 núcleo · 16 MHz	2 núcleos Xtensa LX6 · hasta 240 MHz
SRAM	2 KB	520 KB
Flash	32 KB, dentro del chip	4 MB, chip aparte (SPI)
Tensión lógica	5 V	3,3 V
Pin ↔ periférico	fijo (PB5 = bit 5 de PORTB)	configurable (GPIO matrix)



Mis apuntes

No cambia: un pin se controla escribiendo bits en **registros mapeados en memoria.**

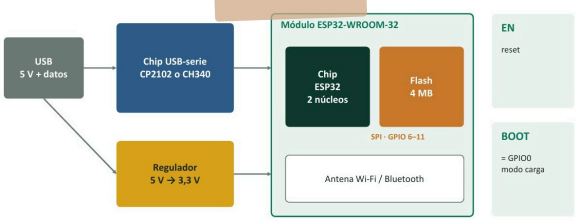
✦ GPIO matrix



AVR: cableado fijo · ESP32: central configurable

Configurar un pin 1. Elegir su función (IO_MUX). 2. Conectar la señal en la **matrix**. 3. Habilitar la salida. `pinMode()` hace los tres.

✦ La placa DevKit V1 (30 pines)



El regulador baja los 5 V del USB a 3,3 V

USB	chip USB-serie (CP2102/CH340): carga y Monitor Serie.
WROOM	ESP32 + flash 4 MB (GPIO 6–11) + antena.
EN	reset.
BOOT	GPIO0, modo de carga.
Ojo	los clones cambian el orden de pines.

DDR, PORT y PIN tienen equivalentes

Función	AVR	ESP32 (GPIO 0–31)
Dirección (entrada/salida)	DDRB	GPIO_ENABLE_REG
Escribir la salida	PORTB	GPIO_OUT_W1TS_REG / GPIO_OUT_W1TC_REG
Leer la entrada	PINB	GPIO_IN_REG
Pull-up interna	PORTB (pin como entrada)	registro IO_MUX del pin

GPIO 32–39 usan un **segundo banco** (`GPIO_OUT1_REG` , `GPIO_IN1_REG`).

El riesgo de leer, modificar y escribir



Una interrupción llega en medio de in → eor → out

1. **in**: copia PORTB (0010 0000).
2. La interrupción pone PORTB = 0010 0001.
3. **eor**: calcula con la *copia vieja*.
4. **out**: escribe 0000 0000.

Resultado: se perdió el cambio de la interrupción. Con **dos núcleos** el riesgo es mayor.

W1TS y W1TC

W1TS write 1 to set
1 → se **ENCIENDE** · 0 → no se toca

W1TC write 1 to clear
1 → se **APAGA** · 0 → no se toca

GPIO_OUT antes	0	1	0	1
escribo en W1TS	0	0	1	0
GPIO_OUT después	0	1	1	1

Con W1TC: antes 0111, escribo 0010, después 0101.

Una sola escritura, sin leer → operación **atómica**, aunque haya interrupciones u otro núcleo.

Blink con registros del ESP32

```cpp
#include "soc/gpio_reg.h"
constexpr uint8_t  PIN_LED  = 16;
constexpr uint32_t MASK_LED = 1UL << PIN_LED;

void setup() {
  pinMode(PIN_LED, OUTPUT);      // configura
}
void loop() {
  REG_WRITE(GPIO_OUT_W1TS_REG, MASK_LED); // PORTB |=
  delay(500);
  REG_WRITE(GPIO_OUT_W1TC_REG, MASK_LED); // PORTB &= ~
  delay(500);
}
```

Montaje

LED en GPIO16
330 Ω a GND
ESP32 Dev Module
Core Espressif 3.x

Capas de abstracción



Arriba: más fácil · abajo: más control y velocidad

La API cuesta tiempo.

digitalWrite() → ESP-IDF → registro → pin a 3,3 V.

Mapa de pines del DevKit V1



DevKit V1, vista superior

- **Seguros** para LEDs y pulsadores
- **Strapping**: se leen al arrancar
- **Solo entrada**, sin pull-up
- **UART0**: carga y Monitor Serie
- **Alimentación** y reset

AP2: LEDs en 16, 17, 18, 19 y pulsador en 21. Los pines 6–11 (flash) no salen en la placa.

Pines de strapping

GPIO0	GPIO2	GPIO12
En **BAJO** al arrancar → modo de carga (botón **BOOT**).	En bajo o al aire para cargar. Tiene el LED azul de la placa.	En **ALTO** al arrancar → flash a 1,8 V: no arranca.

GPIO5 y GPIO15 también se leen al arrancar. **Regla:** no conectar entradas a pines de strapping.

Wokwi NO reproduce estas fallas.

3,3 V: límites eléctricos

$$R = (3{,}3\ V - Vf) / I$$

LED rojo, 330 Ω → (3,3 – 2,0) / 330 ≈ **4 mA**

Diseño: **≤ 10 mA** por pin · máximo **3,3 V** en una entrada

No toleran 5 V

Un módulo de 5 V directo a una entrada puede **dañar el pin**.

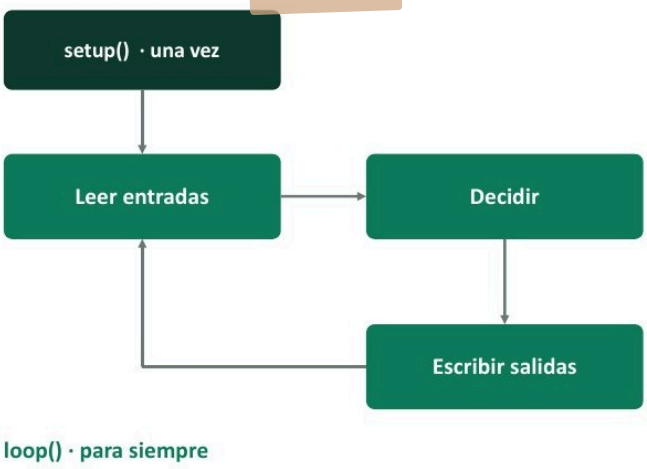
LEDs

Rojo, verde o amarillo con 220–330 Ω. Azul y blanco casi no encienden.

Programa de PC vs embebido

PC: lo lanza el SO y `main()` termina con `return 0`. **Embebido:** arranca con la energía, configura una vez y repite para siempre; no existe un «fin del programa».

El superlazo: setup() y loop()



Ciclo del superlazo: leer, decidir, escribir

Lo que Arduino esconde (AVR):

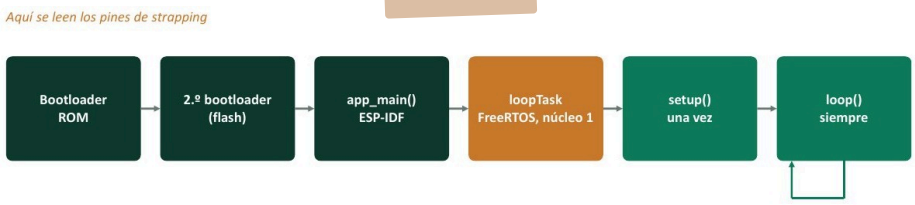
```
int main() {
  init();   // timers, ADC...
  setup();
  for (;;) {
    loop();
  }
}
```

En el ESP32, **loopTask** hace lo mismo. Un `loop()` vacío sigue activo a 240 MHz (decenas de mA); en **deep sleep**, µA.

Antes de `setup()`: las globales inicializadas pasan de **flash** a **SRAM**.

Todo sketch del ESP32 corre sobre **FreeRTOS** (Unidad 2).

Del reset a loop()



Cadena de arranque: del reset al superlazo

¿Qué imprime en la 3.ª pasada?

```
int contadorGlobal = 0;
void setup() { Serial.begin(115200); }
void loop() {
  int contadorLocal = 0;
  static int contadorStatic = 0;
  contadorGlobal++; contadorLocal++; contadorStatic++;
  Serial.print("global="); Serial.print(contadorGlobal);
  Serial.print(" local=");  Serial.print(contadorLocal);
  Serial.print(" static="); Serial.println(contadorStatic);
  delay(1000);
}
```

Pasada	glob.	local	static
1.ª	1	1	1
2.ª	2	1	2
3.ª	3	1	3

Global y static conservan su valor; la local se reinicia en cada pasada.

Instalar el core ESP32 (Arduino IDE 2.x)

1 Preferencias
Agregar la URL del gestor de placas de Espressif.

2 Gestor
Instalar «esp32 by Espressif Systems» 3.x.

3 Placa
Elegir «ESP32 Dev Module».

4 Puerto
COMx o /dev/ttyUSB0 · driver CP210x o CH340.

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

Cuando el programa no sube

Síntoma	Causa probable	Solución
No aparece ningún puerto	Cable solo de carga o falta driver	Otro cable · driver CP210x/CH340
Wrong boot mode / Timed out	No entró en modo de carga	Mantener BOOT en «Connecting…»
Access denied	Puerto ocupado	Cerrar el otro Monitor Serie
Caracteres basura	Velocidad distinta	115200 en `Serial.begin` y Monitor
Brownout detector	Alimentación débil	Otro puerto USB o cable
No arranca con el circuito	Pin de strapping forzado	Mover a un pin seguro

Resumen rápido

- **Registros:** DDR/PORT/PIN → ENABLE / OUT (W1TS, W1TC) / IN.
- **Atomicidad:** W1TS y W1TC escriben sin leer.
- **Pines:** seguros · strapping · solo entrada · UART0 · 3,3 V.
- **Programa:** setup() una vez, loop() siempre, sobre FreeRTOS.